

The Ethics of Pegasus Spyware and State-Sponsored Surveillance in the Interest of National Security

Roman Marquez-Twisdale
r.marqueztwisdale@digipen.edu

ACM Reference Format:

Roman Marquez-Twisdale. 2026. The Ethics of Pegasus Spyware and State-Sponsored Surveillance in the Interest of National Security. In . ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

1 Introduction

Over the past few years, generative AI has started to become a normal part of software development. Tools that can generate code from simple prompts are now used by students, indie developers, and even large companies working on commercial software. On the surface, this seems like a huge improvement—developers can work faster and solve problems more efficiently. Because of this, it's not surprising that more teams are starting to rely on AI-generated code in real-world applications. However, just because something is useful doesn't automatically mean it's ethical, especially when that code is being shipped in commercial products. One

of the biggest issues isn't just the existence of AI-generated code, but how it's being used. In many cases, developers might not fully understand the code that the AI produces, yet still include it in finished products. This creates a situation where responsibility becomes unclear—if something breaks or introduces a security issue, it's harder to know who is accountable. At the same time, there's also the problem of transparency. If developers don't acknowledge when or how AI was used, it raises concerns about authorship. These problems become more serious in commercial settings, where software affects real users. There are also broader concerns about where AI-generated code comes from. Since these systems are trained on large amounts of existing code, including open-source projects, it's possible that they reproduce patterns or structures without proper acknowledgement to the original authors. This paper argues that using generative AI to generate code for commercial applications is only ethical under certain conditions. Specifically, developers need to understand the code they use and be transparent about how AI contributed to the final product.

2 Background and Stakeholders

To understand the ethical issues around AI-generated code, it helps to clarify what is being discussed and who is affected. In this context, software engineers range from individual developers working on small projects to large teams building complex systems. Generative AI refers to tools that produce

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, Washington, DC, USA

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM

<https://doi.org/10.1145/nnnnnnnn.nnnnnnnn>

code from natural language prompts, often trained on large datasets that include publicly available code and documentation. Commercial applications include not just software sold directly, but also internal tools companies rely on. The ethical concern arises because AI-generated code exists in a gray area. It is not fully written by the developer, but it is also not clearly attributable to any single original source. Research has shown that generative AI systems can reproduce patterns from training data, raising concerns about copyright, ownership, and attribution [2], [7]. These systems are also known to produce inaccurate or misleading outputs, sometimes referred to as “hallucinations,” which can make their use risky in real-world applications [3], [5]. Several key stakeholders are affected. Software engineers benefit from increased productivity but also face risks if they rely on outputs they do not fully understand. Studies show that users often overtrust AI-generated content, which can lead to errors being overlooked [3]. Companies gain efficiency but also take on legal and ethical risks, especially if AI-generated code incorporates copyrighted material without attribution [9]. End users are also affected, even if indirectly. They depend on software to function correctly and securely, so bugs or vulnerabilities can directly impact them. The broader impact of unreliable AI-generated content has already been seen in other domains, where it creates challenges for maintaining quality and trust [10]. The open-source community is another important stakeholder. Many AI systems are trained on publicly available repositories, often without explicit consent from contributors. This raises concerns about fairness and respect for intellectual labor, especially when outputs are used in commercial products [7], [9]. Finally, the organizations that develop AI systems also play a role, as ethical guidelines emphasize transparency, accountability, and fairness in system design [4], [8].

3 Current State of Generative AI

Generative AI has quickly become a common tool in software development. Many developers now use these tools for tasks like writing boilerplate code, debugging, and documentation. Studies show that a large majority of developers use generative AI regularly, and that it can significantly reduce the time required for routine programming tasks [6]. One of the biggest advantages is the productivity boost. Developers can generate functional code much faster than writing everything from scratch, especially for repetitive tasks. Research suggests these tools are particularly effective for documentation, testing, and basic implementation [6]. However, these benefits come with limitations. AI-generated code is not always correct and can contain logical errors or subtle bugs. Even when it appears correct, it may include poor design choices. This is because generative AI predicts patterns rather than truly understanding the code [3], [5]. Security is another concern. Research has found that AI-generated code can include unsafe practices, and many developers do not consistently review it before use [6]. There is also the issue of over-reliance—developers may use AI outputs without fully checking them, even if they don’t completely trust them. Over time, this can also affect skill development, as developers rely more on AI instead of problem-solving themselves [6]. Overall, generative AI is powerful but still imperfect. It works well for simple tasks but often struggles with more complex problems, making human oversight necessary.

4 Technical Background

Most generative AI tools are based on large language models trained on massive datasets of text and code. These models generate output by predicting the next token in a sequence based on probability rather than true understanding [3], [6]. Because of this, AI-generated code can look correct while

still containing hidden issues. The output is influenced by training data, which raises concerns about originality and copyright [7]. The model is not intentionally copying code, but it also cannot ensure that its output is completely original. Another limitation is that the model cannot verify correctness or security. It may generate code that compiles but behaves incorrectly or includes vulnerabilities. Research has shown that AI-generated content can include inaccuracies or “hallucinations,” which can be especially risky in software development [5].

5 Ethical Analysis

Social Contract Theory provides a useful framework for evaluating this issue. It is based on the idea that society functions through shared expectations like honesty, fairness, and accountability. In software development, there is an implicit agreement that developers understand and take responsibility for the code they produce. When developers rely on AI-generated code without fully understanding it, this weakens trust. If something goes wrong, responsibility becomes unclear, which conflicts with the expectation that developers stand behind their work. Transparency is another key part of this social contract. If developers do not acknowledge AI use, they may present the work as entirely their own, raising concerns about fairness and authorship. This is especially important given that AI systems are trained on other developers’ contributions [7], [9]. There is also an implicit agreement with the open-source community. Contributors share their work expecting proper credit, but AI systems may produce similar code without attribution. This creates tension between convenience and fairness. However, Social Contract Theory does not imply that AI is always unethical. If developers review and understand the code they use, and if companies are transparent about AI involvement, they can still uphold these expectations. Ethical guidelines for AI emphasize transparency and accountability as key principles [4], [8].

6 Safeguards and Responsibilities

Given the current state of generative AI, the main issue is how it is used. Making AI-generated code ethical comes down to safeguards and responsibility. One of the most important safeguards is requiring developers to review and understand any AI-generated code before using it. Since AI can produce incorrect or misleading outputs, blindly trusting it can lead to bugs or security issues [3], [5]. Transparency is also essential. Developers and companies should acknowledge when AI tools are used, especially in commercial projects. Without this, it becomes harder to assign responsibility or evaluate originality [4], [8]. Companies also have a role in setting standards, such as requiring code reviews and training developers on risks. Since they benefit from AI, they are responsible for managing its impact. Finally, the organizations that develop AI systems share responsibility for how models are trained and what data is used. Concerns about bias, copyright, and fairness suggest that these systems should be designed with ethical risks in mind [4], [8].

7 Conclusion

Generative AI is becoming a normal part of software development. It offers clear benefits in terms of productivity and accessibility, but that does not mean its use is always ethical. The main concerns are lack of understanding and lack of transparency. When developers use AI-generated code without fully understanding it, they take responsibility for something they did not completely create. When they fail to acknowledge AI use, it raises concerns about honesty and authorship. From an ethical perspective, developers still have a responsibility to ensure their code is correct and safe, regardless of how it was created. At the same time, misuse of AI could reduce trust in software systems and create unfair conditions for others [2], [8]. That

said, generative AI can be used ethically if certain conditions are met. Developers need to understand and verify the code they use, companies need clear guidelines, and there needs to be transparency about AI's role. Without these safeguards, the risks can outweigh the benefits, especially in commercial applications. For me personally, this changes how I think about using AI tools in my

own work. It's easy to rely on them for quick answers, but that doesn't mean I actually understand what I'm shipping. Going forward, I think the most important takeaway is that using AI responsibly means treating it as a tool, not a replacement for understanding. If developers can do that, then AI can be a helpful part of the development process. If not, it becomes a shortcut that creates more problems than it solves.